

API SECURITY RISK ANALYSIS REPORT

**A security risk assessment of public API endpoints
to identify vulnerabilities and recommend
security improvements.**

TARGET API : DummyJSON Public API

PREPARED BY : K.KANISHKA

**COURSE / DEPARTMENT : B.E ELECTRONIC COMMUNICATION
Engineering**

DATE OF SUBMISSION : 06/06/2026

TABLE OF CONTENTS

1. Introduction

2. Objective

3. Scope of Assessment

4. Tools Used

5. Methodology

6. API Endpoints Tested

7. Risk Findings and Analysis

7.1 Excessive Data Exposure

7.2 Missing Authentication

7.3 Lack of Rate Limiting

7.4 Weak Input Validation

8. Risk Severity Summary

9. Remediation Recommendations

10. Conclusion

11. References

INTRODUCTION

Application Programming Interfaces (APIs) are essential components of modern web applications, mobile applications, and SaaS platforms. APIs allow communication and data exchange between different systems, enabling services such as user authentication, payment processing, cloud integration, and data sharing. Due to their widespread use, APIs have become a major target for cyberattacks and security exploitation.

Insecure APIs can expose sensitive user information, weaken authentication systems, and create vulnerabilities such as unauthorized access, excessive data exposure, broken access control, and denial-of-service attacks. Poorly secured APIs may also affect business operations, customer trust, and overall system security.

This report presents an API Security Risk Analysis conducted on a public demo API using Postman, Browser DevTools, and API testing techniques. The assessment focuses on identifying common API security weaknesses, evaluating their potential business impact, and recommending remediation measures to improve the overall API security posture. The analysis includes authentication review, authorization assessment, input validation testing, rate-limiting analysis, and data exposure evaluation.

OBJECTIVE

The objective of this API Security Risk Analysis is to identify potential security weaknesses in public API endpoints and evaluate risks related to authentication, authorization, data exposure, input validation, and rate limiting. The assessment aims to analyze how insecure API configurations may impact application security, business operations, and user data protection.

This report also focuses on documenting identified risks, explaining their potential impact in a business-friendly manner, and providing appropriate remediation recommendations to improve the overall security posture of the API environment.

SCOPE OF ASSESSMENT

This assessment focuses on analyzing the security of the DummyJSON public demo API using Postman and Browser DevTools. The assessment included authentication analysis, authorization testing, data exposure evaluation, input validation testing, and rate-limiting analysis.

The testing was limited to publicly accessible demo API endpoints intended for educational and security testing purposes only. Add a little bit of body text

TOOLS USED

The following tools and technologies were used during the API Security Risk Analysis assessment:

1. Postman

Postman was used to send API requests, analyze responses, test authentication mechanisms, and evaluate API endpoint behavior.

2. Browser DevTools

Browser Developer Tools were used to inspect network requests, HTTP responses, headers, and API communication within the browser environment.

3. DummyJSON Public API

The DummyJSON public demo API was used as the testing target for performing security analysis and endpoint assessment.

4. Canva / Microsoft Word

Canva and Microsoft Word were used to design, structure, and prepare the professional security assessment report.

5. GitHub

GitHub was used to upload and manage the final project report and related documentation in a public repository. Add a little bit of body text

METHODOLOGY

The API Security Risk Analysis was conducted using a structured testing approach to identify common API security weaknesses and assess potential risks.

The methodology included the following steps:

1. Selected the DummyJSON public demo API as the testing target.
2. Collected and reviewed publicly accessible API endpoints.
3. Tested API requests and responses using Postman.
4. Analyzed authentication and authorization mechanisms.
5. Evaluated API responses for excessive data exposure.
6. Tested input validation using invalid and abnormal values.
7. Assessed the presence of rate-limiting protections.
8. Documented identified risks, business impact, and remediation recommendations.

API ENDPOINTS TESTED

The following API endpoints were tested using Postman to analyze authentication behavior, data exposure, input validation, and rate-limiting protections. The assessment focused on identifying common API security risks and evaluating the overall security posture of the tested endpoints.

END POINT	METHOD	PURPOSE	AUTHENTICATION REQUIRED	TEST RESULT
/users	GET	Retrieve list of user	NO	200 OK- Returns list of user with personal details
/users/1	GET	Retrieve a specific user	NO	200 OK- Returns user details
/Products	GET	Retrieve list of products	NO	200 OK- Returns product data
/auth/login	GET	user login and get token	NO	200 OK- Returns token
/product/add	GET	Add new product	NO	201 Created - product added

1) GET /users

GET

https://dummyjson.com/users

Send

Body

Headers (15)

Test Results

200 OK

Pretty

Raw

Preview

JSON

```
1 {
2   "users": [
3     {
4       "id": 1,
5       "firstName": "Terry",
6       "lastName": "Medhurst",
7       "email": "terry.medhurst@example.com",
8       "age": 50,
9       "address": {
10        "city": "Glen Ellen",
11        "state": "Kentucky"
12      }
13    }
14  ]
15 }
```

Risk(s):

1

2

Returns user list with personal details without authentication.

1 . Excessive Data Exposure

Description:
The API responses expose unnecessary user-related information such as email addresses, age, address details, and other personal data through publicly accessible endpoints like /users and /users/1.

Impact:
Excessive exposure of user-related information may increase the risk of privacy breaches, phishing attacks, profiling, and social engineering activities.

Severity:
Medium

Recommendation:
Implement response filtering and return only the necessary data fields required for application functionality. Sensitive information should be protected and minimized in API responses.

2) GET /users/1



Risk(s): **1** **2**

Individual user details are accessible without authentication (ID enumeration risk).

2 . Missing Authentication

Description:

Several API endpoints are publicly accessible without requiring authentication or authorization controls. Users can access API resources directly without validating their identity.

Impact:

Missing authentication controls may allow unauthorized users to access sensitive resources, increasing the risk of data leakage and misuse.

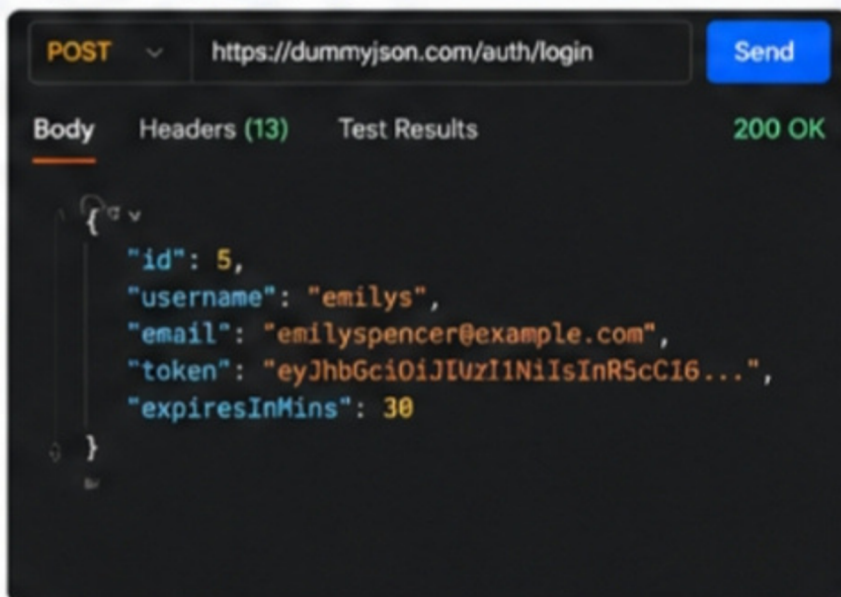
Severity:

Medium

Recommendation:

Implement secure authentication mechanisms such as JWT or OAuth and enforce proper authorization checks for sensitive endpoints.

3) POST /auth/login



Risk(s): **3**

Login endpoint returns token successfully without any rate limiting or brute-force protection.

3 . Lack of Rate Limiting

Description:

The API did not show visible rate-limiting or throttling protections during testing. Multiple repeated requests could be sent without temporary blocking or request restrictions.

Impact:

Lack of rate limiting may allow attackers to perform brute-force attacks, credential stuffing, or abuse API resources, potentially affecting system availability and performance.

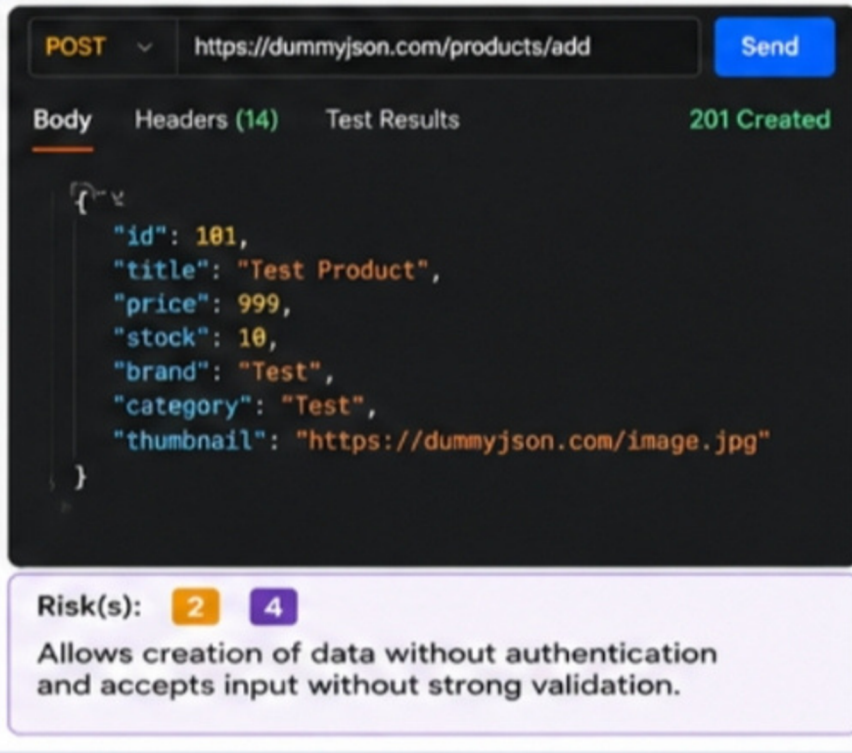
Severity:

Medium

Recommendation:

Implement API rate limiting, request throttling, and monitoring mechanisms to detect and prevent abnormal traffic activity.

4) POST /products/add



4 . Weak Input Validation

Description:

The API accepted abnormal or unvalidated input values during testing, indicating weak server-side input validation controls.

Impact:

Weak input validation may increase the risk of injection attacks, malformed data processing, and application instability.

Severity:

Low to Medium

Recommendation:

Validate and sanitize all user inputs on the server side and enforce strict input validation policies for all API requests. Add a little bit of body text

Analysis and Findings

- Most API endpoints are publicly accessible and do not require authentication.
- Sensitive user-related data is exposed in API responses.
- Potential user enumeration risk exists in endpoints like /users/1.
- Rate limiting protections were not observed during testing.
- The /auth/login endpoint returns authentication tokens without visible request throttling or account lockout mechanisms.

RISK FINDINGS AND ANALYSIS

7.1 Excessive Data Exposure

The API exposed unnecessary user-related information through publicly accessible endpoints. This may increase privacy and security risks.

Severity: Medium

7.2 Missing Authentication

Some API endpoints were accessible without authentication controls, allowing unrestricted access to API resources.

Severity: Medium

7.3 Lack of Rate Limiting

The API did not show visible rate-limiting protections during testing, which may allow request abuse and brute-force attacks.

Severity: Medium

7.4 Weak Input Validation

The API accepted abnormal input values during testing, indicating weak server-side validation controls.

Severity: Low to Medium

RISK AND SEVERITY

- Excessive Data Exposure — Medium Severity
- Missing Authentication — Medium Severity
- Lack of Rate Limiting — Medium Severity
- Weak Input Validation — Low to Medium Severity

Overall Risk Level: Medium

REMEDIATION RECOMMENDATIONS

- Implement secure authentication and authorization controls.
- Limit unnecessary exposure of user-related data in API responses.
- Apply API rate limiting and request throttling mechanisms.
- Validate and sanitize all user inputs on the server side.
- Monitor API traffic and detect abnormal activities regularly.

CONCLUSION

The API Security Risk Analysis identified several common API security weaknesses, including excessive data exposure, missing authentication, lack of rate limiting, and weak input validation controls. Although the tested API is a public demo environment, the identified observations demonstrate how insecure API configurations may create security risks in real-world applications.

Implementing proper authentication, secure access controls, input validation, and rate-limiting protections would significantly improve the overall security posture of the API environment.

REFERENCES

- **OWASP API Security Top 10**
- **DummyJSON API Documentation**
- **Postman Learning Center**
- **GitHub**